Depth Charge: Submarine Defense with Sonar Detection

EL-2003 COAL Lab — Fall 2026 | Team report draft — review and personalize before submission

EL-2003 Computer Organization and Assembly Language Lab — Fall 2026

FAST-NUCES Karachi, BCS-3C

Muhammad Anas (25K-0899), Syed Alyan Hussain (25K-0648), Ali Raza (25K-0827)

Lab Instructor: Mr. Muhammad Owais

> ***Draft for academic review.** This IEEE-style team report was prepared with AI assistance from the repository proposal, code, and course outline. Each member must verify the claims, add personal understanding and real lab-PC evidence, follow the instructor's individual-report rule, and disclose AI use. Do not submit this draft unchanged as unaided individual work.*

Abstract

Depth Charge is a single-player, text-mode submarine defense game implemented in 32-bit x86 assembly using MASM and the Irvine32 library. The player moves a submarine, fires torpedoes, and avoids falling depth charges while a sonar mechanic temporarily reveals hidden mines. The project applies course topics including data definition, registers, addressing modes, arrays, structures, conditional processing, procedures, stack usage, shifts, multiplication/division, string instructions, macros, and operating-system services. The design uses a simple state-driven game loop, fixed-size object arrays, a four-byte object structure, changed-cell console rendering, lookup-table difficulty, short Windows Beep effects, and a four-byte high-score file. The integrated source was assembled and linked as a Win32 console executable in a compatibility environment and passed a 1,090-check procedure harness. Native Visual Studio/lab-PC acceptance, audible sound verification, and the final long-run test remain pending. The result is a complete but deliberately small course project whose modules can be explained individually in a viva.

Keywords— x86 assembly, MASM, Irvine32, game loop, sonar, collision detection, console rendering, COAL lab.

I. Introduction

Introductory assembly exercises often execute once and wait for blocking input. A real-time game cannot stop the whole program while waiting for a key: it must continue updating objects, checking collisions, and drawing frames. Depth Charge uses this problem to connect low-level programming concepts from EL-2003 into one interactive application.

The player controls a submarine near the bottom of an 80 × 25 console area. Depth charges fall from the top. Some are hidden mines that remain logically active but are not drawn during normal play. Sonar freezes object movement briefly and reveals those

mines. Score, lives, levels, pause, sound, menus, and a persistent high score complete the proposed feature set.

The project is intentionally not a graphics engine. It uses ASCII characters, fixed arrays, short procedures, and ordinary comparisons so that each member can trace the code in a debugger and explain it during the individual viva.

II. Course Alignment and Objectives

The uploaded Fall 2026 outline specifies x86 assembly, an IDE, addressing modes, conditional processing, shifts, stacks, strings, and a project viva/report. Table I maps the implemented work to those topics.

Table I. Course-topic evidence

Topic	Project evidence
IDE/build/debug/run	Visual Studio Win32 project; MASM source and debugger guide
Data definition	BYTE/WORD/DWORD variables, strings, lookup tables
Data transfer/flags	MOV, MOVZX, LEA, CMP, TEST, conditional jumps
Addressing	Direct, indirect, indexed, base-index-scale
Arrays/loops	Torpedo, charge, explosion, and screen-buffer loops
Procedures/stack	PROC, CALL/RET, USES, PUSH/POP, LOCAL
Shifts	SHL for colour placement; SHR in random generation
Multiply/divide	Row offsets, LCG, decimal digits, level calculation
String instructions	REP STOSB, REP STOSW, REP MOVSD
Structures	Four-byte GameObject
Macros	mPlaySound register-saving Beep wrapper
OS services	Console I/O, timing, file I/O, Windows Beep

Recursion, inline assembly, extended multiword arithmetic, and the optional REPE CMPSW renderer are not used. The outline lists course topics; it does not require every topic to be forced into one application.

The objectives are: (1) implement non-blocking controls and boundary checks; (2) manage fixed arrays of moving objects; (3) detect same-cell and crossing collisions; (4) separate hidden logical state from visual state; (5) provide score/lives/levels and persistence; and (6) produce explainable member-owned modules and test evidence.

III. Requirements and Scope

The included scope follows the submitted proposal: single-player 80 × 25 colored text console; submarine, torpedoes, charges, hidden mines, explosions; sonar with reveal and recharge; score, three lives, nine levels; menu, instructions, pause, game over; sound; and a high-score file.

Out of scope: graphics mode, sprites, mouse input, multiplayer/networking, music files, saved games in progress, hand-designed levels, BIOS/DOS interrupts, and direct port I/O. Windows user-mode programs use Irvine32 and Windows API equivalents rather than real-mode video memory or speaker ports.

Controls are A/D or Left/Right to move, W to fire, SPACE for sonar, P to pause/resume, and ESC to return to the menu. A visible charge awards 10 points and a hidden mine awards 25. The level increases every 200 points up to 9. A new record is saved at game over; ESC abandons the current round.

IV. System Design

A. State-driven loop

The program has six states: MENU, PLAYING, SONAR, PAUSED, GAMEOVER, and EXIT. The outer main procedure alternates between menu and round. Within a round, each frame performs:

```
HandleInput
  PLAYING: update torpedoes, charges, spawns, explosions;
           check collisions
  SONAR:   update sonar timer only
  PAUSED:  skip object updates
BuildFrame -> Render -> FrameDelay
```

Input and drawing continue during sonar and pause so the player can resume or exit. When the last life is lost, normal play stops and the game-over screen saves and displays the result.

B. Shared object model

Every torpedo, charge, and explosion uses:

```
GameObject = { xPos: BYTE, yPos: BYTE,
               active: BYTE, hidden: BYTE }
```

The structure occupies four bytes. Arrays contain 8 torpedoes, 15 charges, and 5 explosions. A separate five-byte array stores explosion countdowns. active=0 marks a free slot. hidden=1 marks a mine that is invisible during normal rendering but still participates in movement and collision checks.

C. Module division

Anas owns the main/shared data, input, torpedoes, collisions, timing, pause, reset, and integration. Alyan owns console setup, buffers, rendering, HUD, explosions, and sonar. Ali owns random numbers, charges, screens, score/lives/levels, sound, and high-score persistence. One game.asm translation unit includes the .inc modules, preserving a simple build while allowing separate ownership.

D. Timing model

The frame target is 80 ms, approximately 12.5 frames per second. FrameDelay measures elapsed time with GetMseconds and sleeps for the remainder using Delay. If work already exceeded the target, it does not sleep. Windows scheduling, console calls, and blocking Beep can lengthen a frame, so the design is best-effort rather than hard

real-time. Sonar lasts about 2000 ms and then starts a 10000 ms recharge. Pausing during sonar shifts the sonar start timestamp by the paused duration.

V. Implementation

A. Anas: control and rules

HandleInput polls Irvine32 ReadKey; ZF=1 means no key. It handles special arrow keys through virtual-key codes, converts ASCII letters to lowercase, and enforces unsigned column boundaries 1 and 78. FireTorpedo finds a free slot and sets a three-frame cooldown. UpdateTorpedoes moves shots upward and frees slots above the playfield.

CheckCollisions uses an outer ECX loop over torpedoes and an inner EDX loop over charges. A hit requires the same column and either the same row or a one-row crossing. The crossing check prevents adjacent objects from swapping rows without detection. On a hit, both objects become inactive, score is added, and an explosion/sound is triggered. Submarine hits are limited to one per frame to avoid a cluster removing several lives instantly.

InitGame resets score, lives, level, position, cooldowns, sonar/pause variables, frame count, object arrays, and explosion timers. REP STOSB performs the byte clears. The high score is deliberately preserved.

B. Alyan: presentation and sonar

The screen is represented by two 2000-WORD buffers. Each cell stores ASCII in the low byte and colour in the high byte. The byte offset is (row × 80 + column) × 2. BuildFrame paints the next frame in memory. Render compares it with the previous frame and prints only changed cells, avoiding full-screen clears and reducing flicker. The bottom-right cell is never printed because doing so can scroll a console. REP MOVSD copies 1000 DWORDs, equivalent to 2000 WORDs, into the previous buffer.

PutNumber converts a value to five decimal characters using repeated division by ten and a six-byte LOCAL stack buffer. The HUD displays score, lives, level, and sonar status. Explosions use five slots and a five-frame character/colour sequence. StartSonar records the start time; UpdateSonar renders the revealed frame, plays the sweep once, and returns to play after the reveal interval. Hidden mines are drawn as green ! only in the sonar state.

C. Ali: content, rules, and persistence

Rand implements a simple linear congruential generator: multiply the seed, add a constant, shift right, and divide by the requested small range. SpawnCharge uses columns 1–78 and marks about one in five charges hidden. UpdateCharges moves active charges according to a frame-count timer.

AddScore computes score / 200 + 1, capped at 9. ApplyLevel reads fall and spawn delays from nine-entry tables. LoseLife decrements lives and either recenters the submarine or sets game over. Menus use direct console output because objects are not moving, and key flushing prevents old gameplay keys from skipping screens.

Five sound effects use Windows Beep through a small macro that saves general registers. Normal-play tones are short because Beep blocks. The high-score file stores

one little-endian DWORD. Load checks for a missing/short/unreadable file and defaults to zero. Save writes only a higher score, checks the written byte count, closes the handle, and reports failure. Only a successful write sets the new-record flag.

VI. Testing and Verification

Verification is separated into compatibility tests already performed and lab-PC tests still pending.

Table II. Recorded compatibility verification — 3 November 2026

Check	Result
MASM 6.14 assembly of production source	Passed
PE32 console link with Irvine32	Passed
Procedure harness against real game code	1,090 checks, 0 failures
Menu/gameplay/pause console smoke test	Passed under Wine/xterm
Random range, spawn cap, fall timer	Passed in harness
Torpedo cooldown/cap/boundary	Passed in harness
Visible/hidden/crossing collisions	Passed in harness
Score/lives/level cap	Passed in harness
Buffer/HUD/explosion/sonar state	Passed in harness
Input letters/arrows/boundaries/pause/ESC	Passed in harness
High-score round trip and error paths	Passed in harness

The harness is a separate validation artifact, not part of the game executable. It includes repeated range checks, so 1,090 checks should not be described as 1,090 distinct features.

Pending lab evidence: Visual Studio 2022 Debug/Release Win32 builds; native Windows appearance/flicker; audible five-effect test; restart persistence on the lab account; folder-permission behavior; and a ten-minute continuous-play soak test. The acceptance checklist contains blank fields for machine, versions, date, actual result, and evidence. No lab-PC pass is claimed in this draft.

VII. Results and Limitations

The integrated program implements the proposed gameplay and provides a traceable mapping to course concepts. The simple design makes register usage, memory layout, and state transitions visible in the debugger. The sonar mechanic demonstrates the distinction between an object's logical existence and its visual representation.

Limitations are explicit: console timing is not guaranteed; the HUD shows five digits; the random generator is not cryptographic; Beep is blocking and device-dependent; the high-score write is not atomic; and the optional fast renderer is omitted. These choices reduce complexity and viva burden while preserving the proposal's required behavior.

VIII. Teamwork, Ethics, and Conclusion

The repository uses member-owned folders and pull requests. The completion PR integrates missing modules and removes temporary stubs. Each member must review their own code, understand the common loop, run the manual tests, and prepare for individual questions. Progress files distinguish implemented code from pending member review and lab verification.

AI assistance was used for implementation support, review, and documentation, following the repository's existing AI-assisted proposal/guides. The team must disclose this according to the instructor's policy and must not claim unreviewed generated work as unaided individual work. The course outline warns that project plagiarism may result in an F grade.

Depth Charge meets the proposed functional scope with a deliberately small x86 assembly implementation. The remaining submission work is honest lab-PC verification, personal report preparation, and viva practice—not additional unrelated features.

References

[1] FAST-NUCES Karachi, *EL-2003 Computer Organization & Assembly Language Lab Course Description Form*, Fall 2026. Uploaded course outline.

[2] K. R. Irvine, *Assembly Language for x86 Processors*, 8th ed. Pearson, 2019.

[3] K. R. Irvine, “Irvine32 library and book resources.” http://asmirvine.com/

[4] Microsoft, *Microsoft Macro Assembler reference*, Microsoft Learn.

[5] Microsoft, *Beep function (utilapiset.h)*, Microsoft Learn.

[6] R. Nystrom, *Game Programming Patterns*, “Game Loop.” https://gameprogrammingpatterns.com/game-loop.html

[7] Project repository, proposal, member guides, source code, and dated verification record.